Trabajo de Taller IISS 2026 — Iteración 3

API REST (CRUD y comandos) y control termostato

Contexto / Objetivo

Diseñar un **API REST** que combine **CRUD** (de habitaciones) y **comandos / tareas** (*tasks*), con una **mínima lógica de control** (termostato simple) que acciona switches contra un stub. El foco de la iteración está en incorpoar el uso de **REST**, el desarrollo de un **stub del switch** y en incorporar una **lógica mínima de control**, partiendo de la persistencia incorporada en la Iteración 2.

Alcance

Diseño del Producto

- Pensar el diseño del **API REST** distinguiendo dos tipos de operaciones:
  - **CRUD** sobre recursos (las habitaciones) y
  - **Comandos / tareas** (*tasks*) que disparan comportamiento (p. ej. accionar un switch, iniciar/parar el controlador, validar el conjunto de habitaciones).

Prototipo

Desarrollar un **API REST para el CRUD de habitaciones**. Cada habitación tiene **id, nombre, temperatura esperada, idTermostato e idSwitch**, con las operaciones de listado, consulta, alta, modificación (incluida la parcial) y borrado. `idTermostato` e `idSwitch` **identifican** los dispositivos de la habitación; su direccionamiento concreto (el topic MQTT del termostato y la URL REST del switch) proviene de la configuración del sitio (ver material adjunto), no del recurso habitación.

El API debe ofrecer además **comandos** (*tasks*):

- **accionar los switches** (ON / OFF) por REST;
- **iniciar y parar el Controlador** (el componente que ejecuta la lógica de control descrita más abajo);
- **validar la consistencia** del conjunto de habitaciones (que no existan identificadores de termostato o switch **faltantes ni duplicados**).

El diseño de los recursos y el uso de los verbos HTTP —incluido **PATCH**— queda a cargo del equipo, respetando el **Nivel 2 del Modelo de Madurez de Richardson**.

Stub del switch. Crear un servicio REST que funcione como **stub del switch**: alcanza con que **informe en consola** las acciones que recibe (prender / apagar). El control acciona los switches de las habitaciones contra este stub.

Control — mínima lógica (termostato simple). El **Controlador** se comporta como un **termostato simple** (por ejemplo, el de un calefón): ante cada lectura de temperatura de una habitación —recibida por MQTT desde el generador de eventos de la Iteración 2—, se compara con su **temperatura esperada** y se acciona su switch:

- si la temperatura medida está **por encima** de la esperada, se **apaga** el switch;
- si está **por debajo**, se **prende** el switch.

Herramientas y Técnicas

- Describir el API con **OpenAPI 3.0** (editor Swagger).
- Implementar el API con **Spring Boot**, desplegado como **contenedor** integrado al `docker-compose`.
- El API debe cumplir con el **Nivel 2 del Modelo de Madurez de Richardson**.
- El API debe requerir **autenticación** en sus operaciones: adoptar un esquema simple (p. ej. **API key** o **token Bearer**) y **declararlo en el OpenAPI** (`securitySchemes`). No se pide gestión de usuarios ni roles: alcanza con proteger el acceso al API.

Proveer un ejemplo completo de uso del API mediante `curl` (incluido el envío de la credencial de autenticación).

Entregables de la iteración

- Documento de descripción del **API en OpenAPI 3.0** (Nivel 2 de Richardson), con el **esquema de autenticación** declarado.
- **API REST** implementada con **Spring Boot** y desplegada como contenedor en el `docker-compose`: **CRUD de habitaciones** (con modificación parcial, vía `PATCH`) y **comandos** (accionar switches ON/OFF, iniciar/parar el Controlador, validar la consistencia del conjunto de habitaciones).
- **Stub REST del switch** que informa en consola las acciones recibidas.
- **Control termostato simple** operativo: acciona el switch de cada habitación (contra el stub) según su temperatura esperada y la temperatura medida.
- Ejemplo de uso del API mediante `curl` (con la credencial de autenticación).
- **Autenticación** aplicada a las operaciones del API: esquema declarado en el OpenAPI y verificable en el ejemplo `curl`.
- Board Jira con la planificación y el avance llevados en tiempo real.

Mecanismo de entrega

- Tag `version3` en GitHub. Copia `.zip` en Moodle.
- Archivos clave:
  - `/docs/api/openapi.yaml` (definición OpenAPI 3.0)
  - `/src/...` (servicio del API de habitaciones y stub del switch, como módulos Maven)
  - `/docker/docker-compose.yml` (actualizado con el servicio del API y el stub del switch)
  - `/scripts/build.sh` y un script de ejemplo de uso del API con `curl`

- `/README.md` (actualizado a esta iteración, entre otros)
